

Direct Collisional Method of Molecular Dynamics Simulation

*A Senior Project Submitted to
The Department of Physics
College of Natural and Computational Sciences
Addis Ababa University*



By
Getabalew Manegerew

Advisor: Dr. Lemi D.

Addis Ababa, Ethiopia
April 2026

ADDIS ABABA UNIVERSITY
DEPARTMENT OF PHYSICS

The senior project of **Getabalew Manegerew**, titled *Direct Collisional Method of Molecular Dynamics Simulation*, is hereby approved.

Advisor Signature

Date

Department Chair

Date

Dedicated to my family, mentors, and the AAU Physics Department staff for their endless support, as well as to the freely available online tools and platforms that facilitated my undergraduate journey.

Contents

Acknowledgements	iv
1 Introduction	1
2 Methodology	3
2.1 Algorithm	4
2.2 Implementation Parameters	5
3 Result	5
4 Discussion	7
5 Conclusion	8
References	10
Appendices	11
A Fortran Implementation of the Hard Sphere Collision Model	11

List of Figures

- | | | |
|---|--|---|
| 1 | Velocity Distribution of the x component of the sampled particle, at $T = 318$ K. | 6 |
| 2 | $X(t)$ for a heavier particle 100 times more massive than the gas particle, and the smaller mass is taken to be equal to the gas particles in mass. For both, the incident radiation assumes an equal speed of ≈ 1000 times more comparing to the gas particles. | 7 |

List of Tables

1	Computed mean and Theoretical means at a temperature of $T = 343$ K, number density $n = 2.34 \times 10^{24}$. The computation window of $1.7 \mu s$ registered, with a time increment of $dt = 0.1 ps$	6
---	--	---

Acknowledgements

I would like to express my sincere gratitude to my advisor, Dr. Lemi D., for his invaluable guidance, support, and patience throughout the development of this senior project. He devoted considerable time to reviewing the manuscript and suggesting methodological improvements. In addition, I benefited significantly from the use of generative AI tools, including Claude, Gemini, and ChatGPT, which assisted in refining the literature review, debugging the Fortran code, and preparing the manuscript — primarily for grammar review and LaTeX compilation. Their support was valuable in improving the overall quality and technical clarity of this work. I am also deeply grateful to my peers for their thoughtful contributions during our informal discussions. Last but not least, I would like to thank Dr. Mesfin, the course instructor, for his encouragement and for bringing out my best efforts throughout this project.

Direct Collisional Method of Molecular Dynamics Simulation

Abstract

Computationally simulating liquefied or rarefied gases remains an established approach in investigating problems in engineering and physics. This is accompanied by a growing volume of computational models for a given type of problem. The aim of those models is to mimic the phase space of real-world systems in a computational environment in a less expensive way. In this study, a new computational model that blends important features of Direct Simulation Monte Carlo (DSMC) and Molecular Dynamics (MD) is proposed, with the assumption of a hard sphere particle model. Under the study of equilibrium gases, physical parameters such as mean free path and specific heat capacity were found to lie within the same order of magnitude as theoretical analysis. The proposed model can be easily extended to account for external effects such as electromagnetic forces, with a functional working regime in non-equilibrium conditions.

1 Introduction

The velocity distribution of non-relativistic gases follows the well-known Maxwellian distribution under equilibrium conditions [1]. From this classical distribution, widely exploited physical quantities are derived based on the sole theoretical analysis, later to be found in great agreement with experimental results. These include the equation of state of the gas as well as the internal energy of the particles. The main drawback of this approach is the inability to explain the non-equilibrium states of a system, which are often encountered in practice. This is the case when aerospace objects re-enter the atmosphere.

One of the peculiar characteristics of the classical equilibrium distribution is the mean free path (m.f.p.) λ , which is the average distance the gas particles move without a collision. For rarefied gases in equilibrium, this is found to be $\lambda \approx 70$ nm at room temperature [1]. To determine whether a system reached its equilibrium, a Knudsen number is introduced:

$$Kn = \frac{\lambda}{L} \quad (1)$$

Here, L is the important characteristic length of the system i.e., the size of the container. Generally, systems with $Kn > 0.01$ have not reached their equilibrium [2]. Importantly, m.f.p. depends on the number density, pressure, and temperature of the gas [1]. Particularly, λ is inversely proportional to the number density of the particles, hence making non-equilibrium flow important for low-density gases.

Solving Boltzmann's equation for a non-equilibrium system is an almost impossible task due to the collision of the particles, except under special circumstances. Resorting to this

approach is followed by molecular gas simulation. At the heart of this, Direct Simulation Monte Carlo (DSMC) and Molecular Dynamics (MD) approaches are used extensively. The former appeared first in 1963 [3]. Its application in aeronautical engineering is widely exploited. The Molecular Dynamics approach heavily relies on solving Newton's second force equation for each of the particles under study. The interactions of the particles are taken into account, and collisions are treated this way. Essentially, each of the particles is allowed to advance in time, and let their interaction dictate how they should behave.

The need for DSMC is apparent in the hydrodynamics of planetary atmospheres such as the Earth and Mars. However, in this paper, a more direct approach is carried out to investigate ideal gas systems. This method borrowed some elements of DSMC, blended with a molecular dynamics approach. However, dividing the box into a number of cells and idealizing simulated molecules as representative of a large number of real molecules from DSMC, as well as the interaction of molecules from MD, are avoided. The former for its indirect nature, the latter for being more direct. In this sense, direct is used to relate the level of mimic of the exact system.

In this paper's approach, particles are idealized to be hard spheres and each of them moves with a well-defined velocity for a specified time much smaller than the mean free time τ of the system. If the separation between two particles is less than their diameter, a collision happens, leading to a uniform scattering in the CM frame as discussed in the methodology section. Unlike MD, there is no need to solve Newton's equations of motion in each time step. However, the history of the particle in the phase space is still available as in MD.

Although hard sphere models with slight modifications exist in the literature [4], the rigid hard sphere model is the easiest to work with. Mathematically, the potential function $\phi(r)$ can be stated as:

$$\phi(r) = \begin{cases} \infty, & r \leq d \\ 0, & r > d \end{cases} \quad (2)$$

This may sound simpler to implement. In practice, there are $\approx 10^{23}$ particles in physically important systems, leading to a catastrophic computational failure. To circumvent this, a very small system is taken with a number of particles in the order of 10^2 , which is assumed to be one small tile in the whole system. This implies we need to apply a periodic boundary condition.

The underlying problem of this study is to investigate the agreement of the classical Maxwell-Boltzmann distribution with the proposed computational model results. This is achieved by comparing predicted characteristic m.f.p. and certain physical quantities such as the specific heat capacity of the system, after a sufficient number of time passages in the modeled system.

Alongside that, a qualitative description of velocity distribution as well as some distinctive phenomena from radiation physics are introduced to show features of the model.

2 Methodology

Since the model employed in this paper is centered on the collision of two particles, it is important to make an explicit statement on how conservation laws are treated in collisions. From experience, it is usually easier to work on the center of mass frame (CMF) to analyze such collisions. Alongside that, it makes it simpler to set conservation laws in the program than it could be in the Lab frame.

Let us set conventions for velocities and positions in the Lab and CM frames. $\mathbf{R}$ and $\mathbf{V}_c$ are the CM position and velocity, respectively, while $\mathbf{r}$ and $\mathbf{v}_r$ are for relative position and velocity in the lab frame. The numbers in the subscript are for particle identification, as they are observed from the Lab frame. Now, we have the following relation for two colliding particles:

$$\mathbf{r} = \mathbf{r}_2 - \mathbf{r}_1 \quad (3)$$

$$\mathbf{R} = \frac{m_1 \mathbf{r}_2 + m_2 \mathbf{r}_1}{M} \quad (4)$$

M is the sum of the mass of the two particles. The time derivative of the above equations gives us the required velocity relation. Hence we have [5]:

$$\mathbf{r}_2 = \mathbf{R} + \frac{\mu}{m_2} \mathbf{r} \quad , \quad \mathbf{v}_2 = \mathbf{V}_c + \frac{\mu}{m_2} \mathbf{v}_r \quad (5)$$

$$\mathbf{r}_1 = \mathbf{R} - \frac{\mu}{m_1} \mathbf{r} \quad , \quad \mathbf{v}_1 = \mathbf{V}_c - \frac{\mu}{m_1} \mathbf{v}_r \quad (6)$$

$\mu = \frac{m_1 m_2}{M}$ is the reduced mass. Furthermore, based on the above relations, the kinetic energy of the system is the sum of the kinetic energy due to the motion of the CM with mass M and the kinetic energy due to the motion of the fictitious mass μ with the relative velocity $\mathbf{v}_r$. This is demonstrated as follows:

$$KE_T = \frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2 \quad (7)$$

$$= \frac{1}{2} m_1 \left(\mathbf{V}_c - \frac{\mu}{m_1} \mathbf{v}_r \right)^2 + \frac{1}{2} m_2 \left(\mathbf{V}_c + \frac{\mu}{m_2} \mathbf{v}_r \right)^2 \quad (8)$$

$$= \frac{1}{2} M V_c^2 + \frac{1}{2} \mu v_r^2 \quad (9)$$

Since the CM velocity is constant before and after collisions (for zero external force), we only need to make the magnitude of $\mathbf{v}_r$ the same before and after the collision for any elastic collisions, in order to conserve the kinetic energy. Additionally, the momentum of the two-particle system is simply the momentum of the CM, which stays constant and is shown below:

$$\mathbf{P}_0 = m_1 \mathbf{v}_1 + m_2 \mathbf{v}_2 \quad (10)$$

$$= m_1 \left(\mathbf{V}_c - \frac{\mu}{m_1} \mathbf{v}_r \right) + m_2 \left(\mathbf{V}_c + \frac{\mu}{m_2} \mathbf{v}_r \right) \quad (11)$$

$$= M \mathbf{V}_c = \mathbf{P}_f \quad (12)$$

The importance of equations 9 and 12 is apparent in the implemented program. As long as the relative velocity magnitude v_r stays the same, we have the freedom to choose its direction in elastic collisions. So, it is possible to assign any value for $\mathbf{v}_r$ that preserves its magnitude, then infer $\mathbf{v}_1$ and $\mathbf{v}_2$ from 5 and 6, respectively.

In our case, a uniform random number distribution in the range of $(-1, 1)$ is used to generate a unit vector. This is justified because the collision between identical particles follows a uniform scattering angle cross-section in the CM frame [5].

Another important physical consideration is the boundary condition. The physical size of the system is much smaller than practical systems, hence a periodic boundary condition is applied, implying the assumption of a small plate on a bigger system.

2.1 Algorithm

1. Define a vectorial array of $N \times 3$ dimensional position $\mathbf{X}$ and Velocity $\mathbf{V}$, which are used to trace the phase space of N particles in 3 dimensions.
2. Initialize Velocity vectors for each particle, sampled from MBD. For the Position vector, assign random positions in the box.
3. Move each particle in a time step of dt , which corresponds to a distance increment of $d\mathbf{X} = \mathbf{V}dt$. Accordingly, update the Position array.
4. Pick a particle i.e., particle i , then compute the separation distance with another particle j as R_{ij} .

If $R_{ij} < dl$, then collide the particles accordingly.

- Compute $\mathbf{v}_r$, $\mathbf{v}_{cm}$, μ , and unit separation vector $\hat{\mathbf{R}}$
- Pick a normalized 3D vector of random numbers, $\hat{\mathbf{n}}$
- Set $\mathbf{v}_r = v_r \hat{\mathbf{n}}$
- Assign the values of $\mathbf{v}_{i\text{new}}$ and $\mathbf{v}_{j\text{new}}$ using equations 5 and 6, respectively
- Assign $\mathbf{x}_{i\text{new}}$ and $\mathbf{x}_{j\text{new}}$ as follows

$$\mathbf{x}_{i\text{new}} = \mathbf{x}_{i\text{old}} + 0.6dl \hat{\mathbf{R}} \quad (13)$$

$$\mathbf{x}_{j\text{new}} = \mathbf{x}_{j\text{old}} - 0.6dl \hat{\mathbf{R}} \quad (14)$$

5. Do step 4 for the rest of the particles. Avoid double counting of collisions.

6. Register the Position and Velocity of the selected particle.
7. Apply periodic boundary conditions for particles near the boundary.
8. Repeat steps 3 to 7 until the desired time of evolution of the system is reached.

In step 3 we need to make sure dt is much smaller than τ . Longer time steps are prone to miss some collisions. After continually making dt smaller and smaller, there is a point at which no further increase in the collision rate is observed. This is the optimal time step we should have in our program.

An associated computational trap in making dt smaller is that the particles don't get enough time to move apart after collisions happen. Since step 4 depends on separation distance for collision detection, this brings artificial collisions in the next step. Hence equations 13 and 14 are introduced to preclude unintended collisions.

2.2 Implementation Parameters

The parameters used to describe the system under study follow typical atmospheric gases. Number of particle density n , Temperature of the system T , Diameter of the molecules dl , Volume of the box V_{box} , and Mass of the molecules M are taken to be the essential ingredients of the program. Their respective values are given with their units, as found in the program:

- $n = 2.34 \times 10^{24} \text{ m}^{-3}$
- $T = 300 \text{ K}$
- $dl = 4.0 \times 10^{-10} \text{ m}$
- $V_{box} = 6.4 \times 10^{-23} \text{ m}^3$
- $M = 10^{-26} \text{ kg}$

The temperature of the system is where the initial velocity distribution is picked from MBD. Otherwise, the computed temperature from kinetic theory is used in comparing the results of the model with theoretical outcomes. For some of the systems discussed, the volume of the box is altered, hence n shifts too.

3 Result

While running the simulation in each time step, the velocity and trajectory of a specified particle have been tracked. Ergodically, we expect this particle to mimic the velocity distribution of the system over a long-time exposure. Figure 1 shows the result for the x component of the velocity of the particle with its theoretically expected probability density curve at a specific temperature run.

In addition to that, Table 1 compares expectation values of specific heat capacity and energy of the specified particle based on the values of its x-component velocity at $T = 343 \text{ K}$. Also, the characteristic length of the mean free path is addressed.

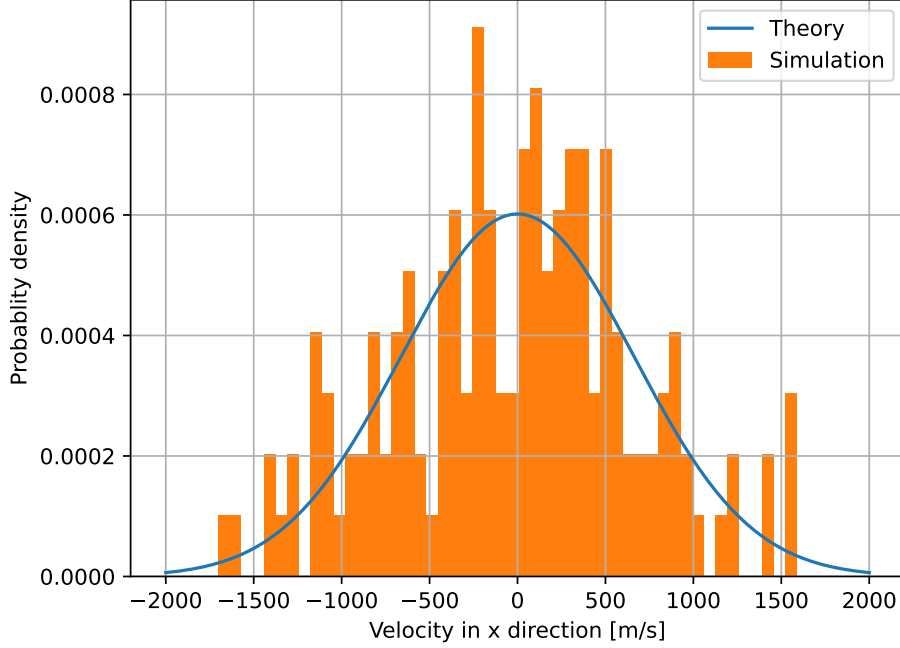


Figure 1: Velocity Distribution of the x component of the sampled particle, at $T = 318$ K.

Quantity	Computed Mean	Theoretical Mean
$\langle E \rangle [10^{-21} \text{ J}]$	1.9269	$\frac{1}{2}k_bT = 2.3665$
$\langle c_v \rangle [10^{-24} \text{ J K}^{-1}]$	5.3879	$\frac{1}{2}k_b = 6.9$
$\lambda [100\text{nm}]$	1.2158	6.0012

Table 1: Computed mean and Theoretical means at a temperature of $T = 343$ K, number density $n = 2.34 \times 10^{24}$. The computation window of $1.7 \mu\text{s}$ registered, with a time increment of $dt = 0.1\text{ps}$.

Another system under study is an energetic particle entering the gas system. This phenomenon arises in nuclear physics, when particle radiation such as protons, neutrons, and beta particles encounter a gas detector. The energy transfer during collisions is highly dependent on the mass of the radiation [6]. The maximum energy transfer for incident particle 1 with initial kinetic energy K_{in} to a particle at rest, particle 2, is given by:

$$T_{max} = \frac{4m_1m_2}{(m_1 + m_2)^2} K_{in} \quad (15)$$

If $m_1 \gg m_2$, energy transfer is insignificant in a single collision. Massive particles like neutrons traverse a straight line, due to the fact that dominant collisions are from atomic electrons. On the other hand, if the colliding particles have nearly equal mass, the energy transfer is efficient. Hence, the particle trajectory is tortuous with fast energy losses. This is the case when the radiation is a beta particle.

Based on the above discussion, we expect varying the mass and kinetic energy of one of the

simulated particles to result in a handful of qualitative features. We expect an undeterred path for a massive particle. Importantly, the equal mass limits show noticeable features, which are easy to consider. Both of those effects are shown in Figure 2.

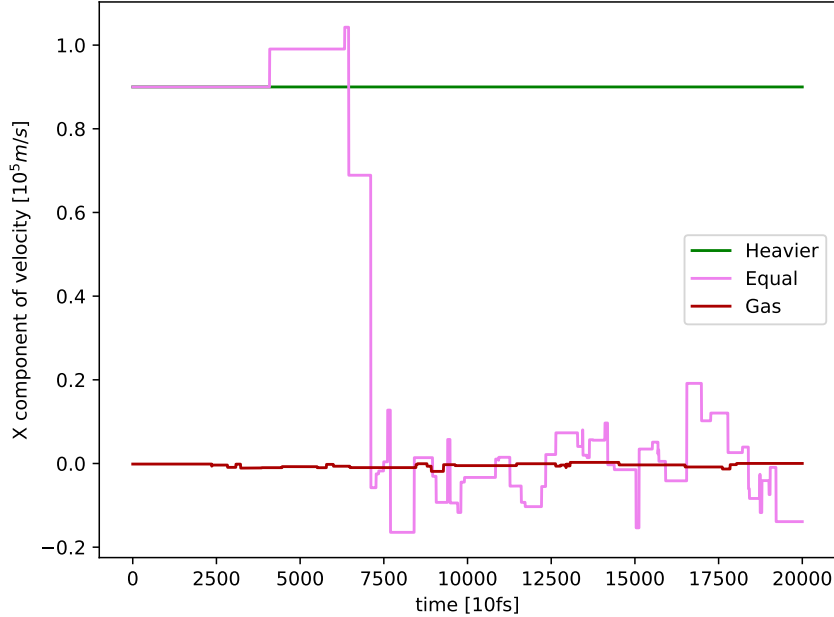


Figure 2: $X(t)$ for a heavier particle 100 times more massive than the gas particle, and the smaller mass is taken to be equal to the gas particles in mass. For both, the incident radiation assumes an equal speed of ≈ 1000 times more comparing to the gas particles.

4 Discussion

The approach implemented in this study marked a significant agreement with theoretical expectations. Particularly, Table 1 shows this quantitatively. Both mean energy and specific heat capacity were found to lie within the same order of magnitude as theoretical values, suggesting the agreement level of our computational model with physical systems.

The important parameter, mean free path, too is confidently in the same order. However, the computational result is observed to be consistently 5 times smaller than the theoretical values. Consequently, a higher number of collisions results than would occur in theoretical analysis. Considering the fact that theoretical results are based solely on the rigid hard sphere, a slight difference is expected. Possibly, velocity variations cause the observed difference. This study does not entirely explain the actual cause of mean free path discrepancy.

The qualitative features of Figure 1 and Figure 2 are what one would expect under normal circumstances. The velocity distribution fits the well-known Maxwell-Boltzmann velocity distribution of the gas. However, there are peaks and dips here and there in the histogram. This is due to the analysis method carried out. A single particle is ergodically expected to assume each of the velocities of the system per probability distribution, after

a sufficient time. However, due to computational constraints, a time bound is set. Hence, only some parts of the velocity distribution are addressed.

On the other hand, the particular system discussed from nuclear physics lends itself to more explanation. The heavier particle's velocity remains unchanged in time, attributed to the fact that energy transfer is insignificant to its initial kinetic energy. The gas particle also shows a relatively uniform velocity, except at some collisional incidents, which is what we normally expect.

Contrary to the previous case, the incident particle with a mass nearly equal to the gas particles exhibits the same pattern initially, until it begins to collide. After the first collision, there is a significant drop in its speed. Consequent collisions bring the system to an energy comparable to the thermal energy of the gas particles. Both of the incident radiation features are what we expect according to equation 15.

The above quantitative and qualitative discussions indicate to what extent our computational model mimics real systems. There is great success in achieving this. Quantitative features remain within a tolerable range of differences. However, the consistent patterns in the variation of those quantities leave room for modifications. Particularly, the energy and specific heat capacity are consistently found to be smaller than the actual values, while the opposite happens for the mean free path. The former are born from a single-particle dependent analysis and are expected to clear up when running the simulation for longer times.

This model accounts only for low-density gases, as each of the particles is taken to be a hard sphere, which is unlikely in a real system. In reality, long-range interactions do have effects. This is one of the most important issues which needs to be addressed in this model.

With a little effort, the model can be extended to include non-isolated systems such as charged particles in an external electromagnetic field. One peculiar example is atmospheric particles in a uniform gravitational field. We are only required to solve Newton's equation of motion explicitly for both the position and velocity of each of the particles independently, then update both of them in each time step accordingly.

5 Conclusion

In this paper, a computational model that blends important features of DSMC and MD is proposed. The aim was to mimic the probability distributions of rarefied gases, while preserving particles history in the phase space. Primarily, a small number of particles, singled out from the whole system, were discussed. Each of those particles is allowed to move with their respective velocity for a specified small time increment dt , which is taken to be much smaller than the mean free time of the system.

A periodic boundary condition is assumed, in order to preserve part of a continuous system assumption. The approach is also heavily based on a hard-sphere particle model, in which interactions beyond the radii of the particles are neglected.

The results of the model are compared with theoretical analysis. All expectations of energy, specific heat capacity, and mean free path of the system were found to lie within the same order of magnitude as theoretical results, which suggests a significant achievement in computational models like this one.

In addition to that, qualitative features of the velocity distribution of the model have taken key elements of the Boltzmann distribution over a long-time exposure. Also, a particular example from radiation physics was treated by taking an incident radiation particle with the desired energy and mass. Importantly, this shows the flexibility of the approach to study such phenomena.

Based on the discussed results, the proposed model reliably mimics practical systems under the specified preconditions. The model could be easily extended to allow an elaborate discussion with systems involving external contacts such as electromagnetic fields. However, the consistent unidirectional shifts of the discussed physical parameters from theoretical results leave room for the enhancement of the model's assumptions and its implementation. A single-particle-dependent analysis of system evolution also affected results under the computational constraint of time.

References

- [1] Stephen J. Blundell and Katherine M. Blundell. *Concepts in Thermal Physics*. Oxford University Press, Oxford, 2nd edition, 2010.
- [2] Iain D. Boyd and Thomas E. Schwartzentruber. *Nonequilibrium Gas Dynamics and Molecular Simulation*. Cambridge University Press, Cambridge, 2017.
- [3] Graeme A. Bird. *Molecular Gas Dynamics and the Direct Simulation of Gas Flows*. Oxford University Press, Oxford, 1994.
- [4] P. S. Prasanth and Jose K. Kakkassery. Molecular models for simulation of rarefied gas flows using direct simulation monte carlo method. *Fluid Dynamics Research*, 40(4):233–252, 2008.
- [5] John R. Taylor. *Classical Mechanics*. University Science Books, Sausalito, CA, 2005.
- [6] James E. Turner. *Atoms, Radiation, and Radiation Protection*. Wiley-VCH, Weinheim, 3rd edition, 2007.
- [7] P. S. Prasanth and Jose K. Kakkassery. Direct simulation Monte Carlo (DSMC): a numerical method for transition-regime flows — a review. *Journal of the Indian Institute of Science*, 86(3):169–192, 2008.
- [8] F. Einar Kruis, Arkadi Maisels, and Heinz Fissan. Direct simulation monte carlo method for particle coagulation and aggregation. *AIChE Journal*, 50(10):2423–2432, 2004.
- [9] Francis J. Alexander, Alejandro L. Garcia, and Berni J. Alder. A consistent boltzmann algorithm. *Physical Review Letters*, 74(26):3512–3515, 1995.
- [10] G. Di Staso, S. Srivastava, E. Arlemark, H. J. H. Clercx, and F. Toschi. Hybrid lattice boltzmann-direct simulation monte carlo approach for flows in three-dimensional geometries. *Computers & Fluids*, 172:492–509, 2018.
- [11] Sohail Murad and Sumnesh Gupta. A simple molecular dynamics simulation for calculating henry’s constant and solubility of gases in liquids. *Chemical Physics Letters*, 319(1-2):60–64, 2000.
- [12] Ioannis Karatzas and Steven E. Shreve. *Brownian Motion and Stochastic Calculus*. Springer-Verlag, New York, 1988.

A Fortran Implementation of the Hard Sphere Collision Model

The complete Fortran 95/2003 code used to execute the granular hard-sphere molecular gas dynamics model and extract statistics (λ , T_{sim} , and velocity tracking distributions) is provided below:

```

1 PROGRAM collision
2   IMPLICIT NONE
3
4   !=====
5   ! parameters
6   !=====
7
8   ! simulation control
9   INTEGER,      PARAMETER :: Np      = 150           ! number of
10      particles
11   INTEGER,      PARAMETER :: D       = 3           ! dimension of
12      the grid
13   REAL(KIND=4), PARAMETER :: t_span = 2e6          ! total time span
14   REAL(KIND=4), PARAMETER :: dt      = 1e2          ! time step
15   INTEGER,      PARAMETER :: Nloop   = t_span/dt    ! number of time
16      steps
17
18   ! physical constants
19   REAL(KIND=4), PARAMETER :: kb      = 1.38e-6      ! boltzmann
20      constant
21   REAL(KIND=4), PARAMETER :: PI      = 4.0*ATAN(1.0) ! pi
22
23   ! grid and particle size
24   REAL(KIND=4), PARAMETER :: l       = 100.0        ! grid size
25   REAL(KIND=4), PARAMETER :: dl      = 4.0          ! particle size
26
27   !=====
28   ! variables
29   !=====
30
31   ! position, velocity, momentum of each particle
32   REAL(KIND=4), DIMENSION(Np, D) :: X, V, P
33   REAL(KIND=4), DIMENSION(Np)    :: M             ! mass of each
34      particle
35
36   ! relative velocity, CM velocity, reduced mass
37   REAL(KIND=4) :: v_r(D), v_cm(D), m_reduced
38   REAL(KIND=4) :: r_hat(D)                       ! unit separation
39      vector
40   REAL(KIND=4) :: rand(D)                         ! normalized

```

```

35      random vector workspace
36
37      ! initial temperature and maxwellian sampling variables
38      REAL(KIND=4) :: T_init = 300.0           ! initial
39      temperature
40      REAL(KIND=4) :: sigma                     ! sqrt(kb*T/m),
41      maxwellian width
42      REAL(KIND=4) :: u1, u2                   ! box-muller
43      uniform samples
44
45      ! time and collision counters
46      REAL(KIND=4) :: t = 0.0
47      INTEGER      :: count = 0                 ! timestep count
48      INTEGER      :: boundary_hit = 0          ! boundary
49      crossing count
50      INTEGER      :: Ncol = 0                  ! collision count
51
52      ! tracer particle arrays (particle 1, x-component)
53      REAL, DIMENSION(Nloop) :: Vx_tracer, Vy_tracer, Vz_tracer, X_tracer
54      , Y_tracer, Z_tracer
55
56      ! output parameters
57      REAL :: Vin, Vf                           ! initial and
58      final mean speed
59      REAL :: Temp_sim                           ! measured
60      temperature
61      REAL :: vol_box, Nd                       ! box volume,
62      number density
63      REAL :: cross_section_area                 !  $\pi d^2$ 
64      REAL :: lambda_theory, lambda_sim, lambda_analysis
65      REAL :: coll_freq_sim, coll_freq_theory
66
67      ! loop indices
68      INTEGER :: i, j
69
70      ! cpu timing
71      REAL :: begin, endt
72
73      !=====
74      ! initialisation
75      !=====
76
77      call cpu_time(begin)
78
79      X = 0.0; V = 0.0; P = 0.0; M = 10.0
80
81      sigma = SQRT(kb * T_init / M(1))

```

```

73
74 ! randomly initializing particle positions and velocities
75 loop1: DO i = 1, Np
76     loop2: DO j = 1, D
77         CALL RANDOM_NUMBER(X(i,j))
78         CALL RANDOM_NUMBER(u1)
79         CALL RANDOM_NUMBER(u2)
80         X(i,j) = -1 + 2*I*X(i,j) ! uniform
81         in [-1, 1]
82         V(i,j) = sigma * SQRT(-2*LOG(u1)) * COS(2*PI*u2) ! box-
83         muller maxwellian
84     END DO loop2
85 END DO loop1
86
87 Vin = SQRT(SUM(V**2)) / Np ! initial
88 average speed
89
90 ! output of initial velocity distribution
91 OPEN(UNIT=9, FILE="initial_v.csv", STATUS="REPLACE")
92 WRITE(9, '(3A20)') "vx", "vy", "vz"
93 DO i = 1, Np
94     WRITE(9, '(3F20.10)') V(i,1), V(i,2), V(i,3)
95 END DO
96 CLOSE(9)
97
98 !=====
99 ! collision kernel
100 !=====
101
102 loop5: DO WHILE (t < t_span)
103
104     ! updating time and positions
105     t = t + dt
106     X = X + V*dt
107     count = count + 1
108
109     ! recording tracer particle
110     Vx_tracer(count) = V(1,1)
111     X_tracer(count) = X(1,1)
112     Y_tracer(count) = X(1,2)
113
114     loop6: DO i = 1, Np
115         loop7: DO j = i+1, Np
116
117             ! looking for approaching particle pairs
118             IF (SUM((X(i,:) - X(j,:))**2) < dl**2) THEN

```

```

117      v_r          = V(i,:) - V(j,:)
                                           ! relative
                                           velocity
118      v_cm          = (M(i)*V(i,:) + M(j)*V(j,:)) / (M(i) + M
        (j)) ! center of mass velocity
119      m_reduced = M(i)*M(j) / (M(i) + M(j))
                                           ! reduced mass
120
121      ! generating normalized random 3D vector for post-
        collision direction
122      CALL RANDOM_NUMBER(rand)
123      rand = 2*rand - 1
124      rand = rand / SQRT(SUM(rand**2))
125      v_r = SQRT(SUM(v_r**2)) * rand
126
127      r_hat = (X(i,:) - X(j,:)) / SQRT(SUM((X(i,:) - X(j
        ,:))**2)) ! unit separation vector
128
129      ! updating velocities and resolving overlap
130      V(i,:) = v_cm + (m_reduced/M(i)) * v_r
131      V(j,:) = v_cm - (m_reduced/M(j)) * v_r
132      X(i,:) = X(i,:) + 0.6*dl * r_hat
133      X(j,:) = X(j,:) - 0.6*dl * r_hat
134
135      Ncol = Ncol + 1
136      END IF
137
138      END DO loop7
139
140      ! imposing periodic boundary condition
141      loop8: DO j = 1, D
142          IF (ABS(X(i,j)) > 1) THEN
143              X(i,j) = X(i,j) - 2*1 * NINT(X(i,j)/(2*1))
144              boundary_hit = boundary_hit + 1
145          END IF
146      END DO loop8
147
148      END DO loop6
149
150      END DO loop5
151
152      call cpu_time(endt)
153
154      !=====
155      ! computing output parameters
156      !=====
157

```

```

158 Vf = SQRT(SUM(V**2)) / Np
159 Temp_sim = M(1) * SUM(V**2) / (3.0 * Np * kb) !
      equipartition temperature
160 vol_box = (2*1)**3
161 Nd = Np / vol_box
      ! number density

162 cross_section_area = PI * dl**2
163 lambda_theory = 0.707 / (cross_section_area * Nd)
164 coll_freq_theory = Vf / lambda_theory
165 coll_freq_sim = REAL(Ncol) / (Np * t_span)
166 lambda_sim = Vin / coll_freq_sim
167 lambda_analysis = (vol_box / (4.0*PI*(dl/2)**3/3.0)) &
168                  * dl / (Np-1) * 0.707 / Vf
169
170 !=====
171 ! output
172 !=====
173
174 ! writing tracer particle trajectory
175 OPEN(UNIT=9, FILE="tracer.csv", STATUS="REPLACE")
176 WRITE(9, '(3A20)') "vx_tracer", "x_tracer", "y_tracer"
177 DO i = 1, Nloop
178     WRITE(9, '(3F20.10)') Vx_tracer(i), X_tracer(i), Y_tracer(i)
179 END DO
180 CLOSE(9)
181
182 ! output of final velocity distribution
183 OPEN(UNIT=9, FILE="final_v.csv", STATUS="REPLACE")
184 WRITE(9, '(3A20)') "vfx", "vfy", "vfz"
185 DO i = 1, Np
186     WRITE(9, '(3F20.10)') V(i,1), V(i,2), V(i,3)
187 END DO
188 CLOSE(9)
189
190 ! printing simulation summary
191 WRITE(*, '(A)') "== simulation summary =="
192 WRITE(*, '(A, I10)') " timesteps : ", count
193 WRITE(*, '(A, F2.15)') " final time[s] : ", t/1e15
194 WRITE(*, '(A, I10)') " boundary crossings : ", boundary_hit
195 WRITE(*, '(A, I10)') " total collisions : ", Ncol
196 WRITE(*, '(A, F12.6)') " Vin, Vf/Vin : ", Vin, " ", Vf/
      Vin
197 WRITE(*, '(A, F12.4)') " T_sim [K] : ", Temp_sim
198 WRITE(*, '(A, ES12.4)') " lambda (theory) : ", lambda_theory
199 WRITE(*, '(A, ES12.4)') " lambda (sim) : ", lambda_sim
200 WRITE(*, '(A, ES12.4)') " coll freq (sim) : ", coll_freq_sim
201 WRITE(*, '(A, ES12.4)') " coll freq (theory) : ", coll_freq_theory

```

```
202     WRITE(*, '(A, F12.4)') "    wall hits/collision: ", REAL(  
        boundary_hit)/Ncol  
203     WRITE(*, '(A, F12.4)') "    collisions/particle: ", REAL(Ncol)/Np  
204     WRITE(*, '(A, F10.3)') "    cpu time [s]          : ", endt - begin  
205     WRITE(*, '(A)') "===== "  
206  
207 END PROGRAM collision
```